

```
kobject_uevent_env(&dev->dev->kobj, KOBJ_CHANGE, uevent_
envp);
pr_info("%s: sent uevent %s\n", __func__, uevent_envp[0]);
```

In the user space, Android's *UsbDeviceManager* collects this *USB_STATE* information and propagates it to the listeners. You can check out the *UsbDeviceManager.java* code that handles this in the repository: <http://bit.ly/R2fCLP>.

After that example, let us write a simple C program to see how to receive uevents in user space.

Receiving uevents in user space

Since the kernel uevent uses a netlink socket, it requires a simple socket program in user space to receive uevent messages. Figure 2 illustrates a simple uevent flow through kernel space to the user space. The best example of how to receive and decode uevents in user space is available as open source (*uevent_listen.c*) and we will use it to explain things.

The first step to receiving a uevent in user space is to connect to or open the netlink socket. In this case, it is *NETLINK_KOBJECT_UEVENT*. This allows us to receive the kernel messages.

```
sock = socket(PF_NETLINK, SOCK_DGRAM, NETLINK_KOBJECT_UEVENT);
if (sock == -1) {
    printf("error getting socket, exit\n");
    exit(1);
}

retval = bind(sock, (struct sockaddr *) &snl,
    sizeof(struct sockaddr_nl));
if (retval < 0) {
    printf("bind failed, exit\n");
    goto exit;
}
```

After successfully binding with the *NETLINK_KOBJECT_UEVENT*, the program should wait, listening to the uevent socket for messages from the kernel. The maximum buffer size of the uevent message is 2048, as defined in */include/linux/kobject.h*, and the receive buffer will be of the same size.

```
buflen = recv(sock, &buffer, sizeof(buffer), 0);
if (buflen < 0) {
    printf("error receiving message\n");
    continue;
}
```

The received message holds multiple string data created by the kernel uevent framework, and parsing can be based on the length returned by the socket API.

```
/* hotplug events have the environment attached - reconstruct
```

```
envp[] */
for (i = 0; (bufpos < (size_t)buflen) && (i < HOTPLUG_NUM_ENVP-1);
i++) {
    int keylen;
    char *key;
    key = &buffer[bufpos];
    keylen = strlen(key);
    envp[i] = key;
    bufpos += keylen + 1;
}
envp[i] = NULL;

printf("[%i] received '%s' from '%s'\n", time(NULL), action,
devpath);

/* print payload environment */
for (i = 0; envp[i] != NULL; i++)
    printf("%s\n", envp[i]);
```

Having seen important blocks of *uevent_listen.c*, let us build and run the program on the terminal. The following snippet shows one of the uevents received when you connect a USB pen drive to your PC while running *uevent_listen.out*:

```
[1345998474] received 'add' from '/devices/
pci0000:00/0000:00:1d.7/usb2/2-1'
ACTION=add
DEVPATH=/devices/pci0000:00/0000:00:1d.7/usb2/2-1
SUBSYSTEM=usb
MAJOR=189
MINOR=149
DEVNAME=bus/usb/002/022
DEVTYPE=usb_device
PRODUCT=cf2/6230/100
TYPE=0/0/0
BUSNUM=002
DEVNUM=022
SEQNUM=2548
```

To understand how the following message was constructed, you can refer to the *usb_dev_uevent* (*drivers/usb/core/usb.c*) and *usb_uevent* (*drivers/usb/core/driver.c*) functions.

This article provided a very brief introduction of the Linux uevent notification mechanism, followed by a simple user space program that receives uevents in user space. There are many more interesting features you can explore like uevent filters, uevent files for cold-plugged devices, udev rules, etc, to improve your understanding of the uevent mechanism. 

By: Rajaram Regupathy

The author works as a Linux specialist with ST Ericsson India Pvt Ltd, and is the author of the book 'Bootstrap Yourself with Linux-USB Stack'.